

谱方法在模型降维中的应用

PB19 [REDACTED] [REDACTED]

2022 年 8 月 8 日

目 录	2
-----	---

目录

1 引言	4
2 模型降维问题的理论基础及主要挑战	5
3 谱方法的理论基础	7
4 约束模态分析	9
5 谱基函数稀疏化 (Sparsifying Spectral Bases)	12
6 向非线性问题拓展	15
7 前沿问题展望	18
8 结论与总结	19

摘要

本文简要总结了一些涉及谱方法在物理模拟的模型降维问题中的应用与发展的论文, 介绍了模型降维与应用谱方法的理论基础, 并从约束模态分析, 谱基函数稀疏化, 非线性问题中的谱方法应用等方面简述了近年来的一些进展, 最后介绍了领域内的一些前沿重点问题. 在本文中, 我们将重点关注这些论文中的相关算法, 并对其做简要概括.

关键字: 物理模拟, 模型降维, 谱方法, 模态分析

1 引言

物理模拟 (Simulation) 是一种在计算机的虚拟世界中重现现实世界中的物理现象的过程, 图形学领域内对此的研究已经进行了多年, 其研究对象包括弹性模拟, 流体模拟, 声音模拟以及一些特殊物理现象 (如磁流体和等离子体) 的模拟等. 随着 VR/AR 技术的快速发展, 以及计算机模拟画面的不断进步, 真实世界的虚拟重现成为一个越来越热门的课题, 这也对物理模拟的精度和性能提出了越来越高的要求. 然而物理方程本身的复杂性使得大多数离散模拟方法在处理大规模问题时性能会急剧下降, 以至实际应用中无法满足用户快速响应的需求. 因此, 对模型系统的降维处理就成为了一个研究热点, 模型降维 (Model Reduction) 也就有了越来越广泛的发展与应用.

谱方法 (Spectral Method) 是一种偏微分方程的数值解方法, 它将解近似地展开成一些光滑函数的级数展开式, 并 (一般地) 使这些光滑函数正交, 得到的结果称为解的近似谱展开式, 再将展开式代入原微分方程组, 就可以得到展开式系数的方程组, 而展开式系数的方程组一般是较易解的. 实际上, 在对解进行谱展开后, 可以直观地看到, 谱方法就是标准的分离变量技术的一种推广. 一般, 在采用谱方法时取切比雪夫多项式和勒让德多项式作为近似展开式的基函数; 对于周期性的边界条件, 则一般采用傅里叶级数和面调和级数比较方便. 谱方法的精度, 直接取决于级数展开式的项数.

物理模拟的本质仍然是数值求解一些偏微分方程组, 因此, 作为一种广泛应用的偏微分方程数值解方法, 谱方法也是一种目前而言广泛应用的模型降维思路, 并且被引入图形学领域的时间很早. 早在 1989 年, Pentland 等人 [1] 就已经使用了这一方法. 他认为, 可以单独表示系统动力学的振动模态, 然后利用多项式变形映射 (Polynomial Deformation Mapping) 将系统动力学的振动模态与系统几何体的体积及模型耦合. 其中这些振动模态表示为一些矩阵的特征向量.

2 模型降维问题的理论基础及主要挑战

物理模拟的基础, 仍然是 Newton, Lagrange, Hamilton 等人发展的力学理论及由其导出的各类偏微分方程 (以下简称 PDE). 这些 PDE 再加上边值条件与初值条件, 即可以预测一个系统的未来演化. 但仅有极少的 PDEs 可以求出解析解, 大多数的 PDEs 只能通过数值方法求出其数值解.

求解 PDEs 的数值方法存在很多挑战, 比如维度爆炸问题: 在提高精度时不可避免地要细分网格, 从而增大计算机的存储量和计算量, 造成不可接受的存储花费与时间花费; 比如非线性, 非凸性问题, 这些问题的求解算法通常需要大量迭代, 且不容易收敛; 再比如一些低条件数问题: 这些问题通常由网格的剧烈变化, 或是材质的突然改变而引起, 会使得结果对输入误差极度敏感; 还有适应性问题: 很多的工作其适应性都很差, 仅能在一个很小的范围中起效, 而很难复现或用于其他问题之中. 这些问题都对 PDEs 的数值解方法提出了极大的挑战.

模型降维是对以上问题的一个行之有效的解决思路. 它意味着将连续的 PDE 转化为离散的形式, 此时, 就需要将各类微分算子 (如 $\nabla, \Delta, \nabla \cdot, \frac{d}{dt}$ 等) 进行离散化的表示. 一些比较经典的工作有对坐标求偏微分时使用的有限差分表示, 以及对时间求微分时使用的 Euler 公式与 Runge-Kutta 算法等等. 这样就可以得到一个离散的代数方程

$$A(x_t) = b_t$$

或是 (在非线性情况下) 得到其迭代表示

$$A'(x_t)\Delta x = b_t - A(x_t)$$

, 迭代求解直至其收敛.

在这种情况下, 选择的这些离散化表示方法本质上是选择一组基函数 $\phi_{i=1}^n$, 对一个 PDE

$$\mathcal{L}u = g$$

, 利用其弱形式

$$L(u, v) = G(v)$$

, 其中 $L(u, v) := \int u \cdot \mathcal{L}v, G(v) := \int gv$, 从而写出一个线性方程组

$$A_{ij}u_j = b_i$$

, 其中 $A_{ij} = L(\phi_i, \phi_j)$, $b_i = G(\phi_i)$, 以这个线性方程组的解去逼近 PDEs 的解.

当选定基函数之后, 对任意一个空间内的函数 $u(x)$, 可以将其表示为 $u(x) = \sum_i q_i \phi_i(x)$, 对应地也就有 $\nabla u(x) = \sum_i q_i \nabla \phi_i(x)$ 等表示. 这样还可以对一些较为高维的 PDE 或 ODE 做进一步的简化.

由上可以看到, 模型降维技术的核心问题就是如何选择一组合适的基函数.

选择基函数有三个原则: 第一是近似的准确性, 我们希望基函数可以使得随着基函数个数增加, 近似的结果以指数速度收敛, 这样就可以在一个较低维的空间中做出有效的近似; 第二是计算的代价, 这包括预计算的代价, 运行时间的代价, 以及其结果的条件数是否良好; 第三是适应性问题, 一组合适的基函数应该能够适应物体的几何形状, 应该能够适应物理方程, 应该能够适应非线性情况.

满足这几个原则需要基函数综合考虑一些相关的性质, 比如需要考虑基函数的空间支撑范围, 支撑范围的大小将影响运算时相关矩阵的稀疏性, 影响方法的空间适应性, 并与运行时间相关; 其次还需要考虑基函数在频谱上的带宽, 带宽越窄则基函数越光滑, 收敛性越佳; 最后还需要考虑基函数解析的表达形式, 这包括选择连续函数还是非连续函数, 选择低阶多项式还是高阶多项式等问题.

综合而言, 最理想的基函数应当拥有尽量小的空间支撑范围, 这会减少计算代价; 应当在合适的频谱变换后有尽可能窄的带宽, 这可以提高基函数的收敛性; 应当选取连续函数, 这有助于将误差控制在有界范围内; 应当选取尽量高阶多项式的基函数, 这可以较为方便地处理非线性问题.

当然, 一般情况下, 很难找到同时满足这几个条件的函数, 如何平衡基函数的运算代价与模拟效果之间的关系, 就是模型降维问题的主要挑战.

3 谱方法的理论基础

这一方法的思想是非常自然的. 在寻找一个系统中最有效的降维方法时, 最直接的方法就是找到这个系统的一些特征向量, 这些特征向量一般代表的是系统中比较典型的振动模态, 而其中一些低频的模态通常对视觉效果有更大的影响. 在应用中, 就可以求解出一些低频的振动模态, 然后以之为基函数, 在这几个维度上分解我们需要模拟的运动.

一般来说, 一个动力学方程可以被写成

$$M\ddot{u} + D\dot{u} + Ku = f$$

的形式, 其中, M 被称为质量矩阵, D 被称为阻尼矩阵, K 被称为刚度矩阵, 阻尼矩阵一般可以表示成 $D = \alpha M + \beta K$ 的形式, α 和 β 被称为阻尼系数. 这个方程的每一个特征向量都代表着一个振动模态, 每个振动模态对应的特征向量可以通过求解一些形如

$$K\phi_i = \lambda_i M\phi_i$$

的广义特征值方程得到. 那么以这些特征向量 (显然, 由线性代数的知识就可以知道, 这些特征向量必然是正交的) 为基函数, 需要表示的函数就可以化为

$$u(x) = \sum_{i=0}^k z_i \phi_i(x), k \ll N$$

的形式. 分解过后再代入原方程, 原来的二阶微分方程

$$M\ddot{u} + D\dot{u} + Ku = f$$

就可以化为

$$\sum_{i=0}^k (\ddot{z}_i M\phi_i + \dot{z}_i D\phi_i + z_i K\phi_i) = f$$

注意到其中有已知的两个关系 $D = \alpha M + \beta K$ 与 $K\phi_i = \lambda_i M\phi_i$, 代入即得

$$\sum_{i=0}^k (\ddot{z}_i M\phi_i + \dot{z}_i (\alpha + \beta \lambda_i) M\phi_i + z_i \lambda_i M\phi_i) = f$$

. 取 $\psi_i = M\phi_i$, 再对 f 进行分解使 $f = \sum_{i=0}^k \langle f, \psi_i \rangle \psi_i$, 其中的内积取欧氏内积, 即 $\langle f, \psi_i \rangle = f^T \psi_i = \psi_i^T f$, 那么就有

$$\sum_{i=0}^k (\ddot{z}_i + \dot{z}_i (\alpha + \beta \lambda_i) + z_i \lambda_i) \psi_i = \sum_{i=0}^k (\psi_i^T) \psi_i$$

由于各特征向量之间正交, 故上式等价于解方程

$$\ddot{z}_i + (\alpha + \beta\lambda_i)\dot{z}_i + \lambda_i z_i = \psi_i^T f, i = 1, 2, \dots, k$$

. 这样, 只需要解十几个或者几十个方程就可以很好地模拟原方程的解, 这就是谱方法的基本思路.

由于理论的普适性, 这一方法的应用也十分广泛, 在较大规模的物理模拟问题中经常被使用.Schulz 等人 [2] 和李嗣旺等人 [3] 在 2014 年将其运用于时空插值问题中,James 等人 [4] 在 2006 年利用该方法进行了声音模拟,HongYi Xu 等人 [5] 在 2015 年也将其运用于材料设计的模拟.

4 约束模态分析

用以求解特征向量的特征方程有时需要根据具体情况加入一些约束条件. De Witt 等人在 2012 年关于不可压缩流体的模态分析 [6] 中, 在对向量 Laplace 算子 Δ 进行模态分析时加上了散度为 0 与 Neumann 边界条件的两个约束条件, 从而将方程组变为

$$\begin{cases} \Delta \Phi_k = \lambda_k \Phi_k \\ \text{div}(\Phi_k) = 0 \\ \Phi_k \cdot \mathbf{n} = 0 \text{ at } \partial D \end{cases}$$

这一方程组得到了更符合现实中流体流动情况的基函数. 这也是容易理解的, Laplace 算子的基函数不可能与具体区域无关, 否则会使得流体在不同区域中呈现相同的特性. 实际上, 单靠方程 $\Delta \Phi_k = \lambda_k \Phi_k$ 得到的结果为 $\Phi_k(\mathbf{x}) = (e^{\mathbf{x} \cdot \boldsymbol{\xi}_{k1}}, e^{\mathbf{x} \cdot \boldsymbol{\xi}_{k2}}, \dots, e^{\mathbf{x} \cdot \boldsymbol{\xi}_{kn}})^T$, 其中 $|\boldsymbol{\xi}_{k1}| = |\boldsymbol{\xi}_{k2}| = \dots = |\boldsymbol{\xi}_{kn}| = \sqrt{\lambda_k}$. 可以看到, 这个 λ_k 可以是任意正数, 而 $\boldsymbol{\xi}_{kl}$ 实际上也可以取模长为 $\sqrt{\lambda_k}$ 的任意数, 这个结果得到的特征向量与特征值实际上是没有太大意义的, 只有代表实际情况的 $\text{div}(\Phi_k) = 0$ 与 $\Phi_k \cdot \mathbf{n} = 0 \text{ at } \partial D$ 确定后, 才能得到有意义的特征向量与特征值.

一般地, 在一个较简单的区域内, 上述方程组存在解析解. 比如在一个二维的矩形区域 $[0, \pi]^2$ 内, 向量 Laplace 算子的特征函数满足

$$\Phi_{\mathbf{k}} = \frac{1}{k_x^2 + k_y^2} (k_x \sin(k_x x) \cos(k_y y) \mathbf{a}_x - k_x \cos(k_x x) \sin(k_y y) \mathbf{a}_y)$$

, 其中 $\mathbf{k} = (k_1, k_2) \in \mathbb{Z}^2$ 是被称为“矢量波数”(Vector Wave Number)的整数组. $\Phi_{\mathbf{k}}$ 就是向量 Laplace 算子的基函数, 其对应的特征值为 $\lambda_{\mathbf{k}} = -(k_1^2 + k_2^2)$.

在以这种形式确定了基函数的情况下, 流体的速度场 (Velocity Field) 就可以表示为这些基函数的线性组合, 即

$$\mathbf{u} = \sum_i^N \omega_i \Phi_i$$

一个 N-S 方程在涡度形式下一般形如

$$\dot{\omega} = \text{Adv}(\mathbf{u}, \omega) + \nu \Delta \omega + \text{curl}(\mathbf{f})$$

, 其中 ω 是 $\mathbf{u}$ 的旋度, 即 $\omega = \text{curl}(\mathbf{u})$, ω 代表着流体的涡度场 (Vorticity Field), 而 $\text{Adv}(\cdot, \cdot)$ 为平流算子, 平流项 $\text{Adv}(\mathbf{u}, \omega) := \text{curl}(\omega \times \mathbf{u})$.

注意到

$$\omega = \text{curl}(\mathbf{u}) = \text{curl}\left(\sum_i^N \omega_i \Phi_i\right) = \sum_i^N \omega_i \text{curl}(\Phi_i)$$

, 取 $\phi_i = \text{curl}(\Phi_i)$, 则 $\omega = \sum_i^N \omega_i \phi_i$. 实际上, $\{\phi_i\}$ 构成了涡度场的基函数. 另外还要看到

$$\text{Adv}(\mathbf{u}, \omega) = \sum_i^N \sum_j^N \omega_i \omega_j \text{curl}(\phi_i \times \Phi_j) = \sum_i^N \sum_j^N \omega_i \omega_j \text{Adv}(\Phi_j, \phi_i)$$

该 N-S 方程以上述涡度场的基函数做基函数分解, 可以化为

$$\dot{\omega}_k = \mathbf{w}^T \mathbf{C}_k \mathbf{w} + \nu \lambda_k \omega_k + f_k, k = 1, 2, \dots, N$$

. 其中 $\mathbf{w} = (\omega_1, \omega_2, \dots, \omega_N)^T$, $\mathbf{C}_k$ 是一个 $N \times N$ 矩阵, 其第 i 行第 j 列的元素是 $\langle \text{Adv}(\Phi_i, \phi_j), \phi_k \rangle$.

De Witt 等人对这一算法做了复杂度分析. 结果显示, 随着基函数个数的增加, 预计算的计算时间以平方速度增长, 所需的存储空间以线性速度增长. 也就是说, 这一算法的复杂度相当高.

Domain Type	Domain	Basis Dim.	Mesh Ele.	Precomp. (s)	Memory		Runtime			
					$\mathbf{C}_k$ (Mb)	Basis Fields $\dagger$ (Mb)	Advection (ms)	Vel. Reconst (cached) $\ddagger$ (ms)	Vel. Reconst (closed form) (ms) $\ddagger\ddagger$	External Forces $\ddagger\ddagger\ddagger$ (ms)
		N	T	$O(N^2)$	$O(N^2)$	$O(TN)$	$\approx O(N)$	$O(TN)$	$O(TN)$	$O(TN)$
Closed form	3-D Cube	81		3.6	0.05	66	18	26		
	3-D Cube	172		15	0.3	141	37	56		
	3-D Cube	325		55	1.1	267	70	105		
	3-D Cube	540		149	3.3	443	125	172		
Closed form	3-D Cube	81		3.6	0.05		18		30	
	3-D Cube	172		15	0.3	n/a	37		67	
	3-D Cube	325		55	1.1		70		126	
	3-D Cube	540		149	3.3		125		216	
		N	T	$O(TN^2)$	$O(N^3)$	$O(TN)$	$O(N^3)$	$O(TN)$		$O(TN)$
Tets	Head	64	$\approx 24^3$	436	0.9	26	20	10		
	Head	128	$\approx 24^3$	1689	7.2	52	42	20		
Voxels	Armadillo	16	32^3	18	0.01	13	3.5	5		22
	Armadillo	32	32^3	47	0.1	26	7	10		43
	Armadillo	64	32^3	185	0.9	52	15	20		88
	Armadillo	128	32^3	1109	7.8	105	39	41		172
Voxels	Bunny	32	32^3	79	0.11	26	8	10		22
	Bunny	64	32^3	269	0.95	52	17	19		43
	Bunny	128	32^3	1305	7.8	105	40	39		90
	Bunny	256	32^3	6911	62.4	210	130	87		174

图 1: De Witt 等人得到的复杂度分析表 [6]

QiaoDong Cui 等人 [7] 在 2018 年部分地解决了该算法复杂度过高的问题. De Witt 等人已经指出, 在一个比较简单的区域 (Domain), 比如一个长

方体中, 特征方程对应的基函数会具有解析的形式, 具体地说, 是一些三角函数的乘积, 而 QiaoDong Cui 等人对这一问题进行了进一步的计算.

在一个二维平面上的矩形 $[0, \pi]^2$ 上, 当函数满足对 x 的 $\frac{\partial \Phi_x}{\partial x}|_{x=0, \pi} = 0$ 以及对 y 的 $\Phi_y|_{y=0, \pi} = 0$ (即 Neumann 双边边界条件) 时, 最后的基函数 $\Psi = (\Phi_x, \Phi_y)$ 有

$$\begin{cases} \Phi_x(\mathbf{k}) = \frac{k_y}{\eta_{\mathbf{k}}} \cos(k_x x) \cos(k_y y) \\ \Phi_y(\mathbf{k}) = \frac{k_x}{\eta_{\mathbf{k}}} \sin(k_x x) \sin(k_y y) \end{cases}$$

. 其中 $\mathbf{k}$ 与上述定义相同, $\eta_{\mathbf{k}} = \sqrt{-\lambda_{\mathbf{k}}} = \sqrt{k_x^2 + k_y^2}$.

当满足对 x 的 $\Phi_x|_{x=0} = 0, \frac{\partial \Phi_x}{\partial x}|_{x=\pi} = 0$ 以及对 y 的 $\Phi_y|_{y=0, \pi} = 0$ (即 Neumann 单边边界条件) 时, 基函数有

$$\begin{cases} \Phi_x(\mathbf{k}) = -\frac{2}{\pi} \frac{k_y}{\eta_{\mathbf{k}}} \sin(k_x x) \cos(k_y y) \\ \Phi_y(\mathbf{k}) = \frac{2}{\pi} \frac{k_x}{\eta_{\mathbf{k}}} \cos(k_x x) \sin(k_y y) \end{cases}$$

. 但是此时 $k_x \in (\mathbb{Z}^+ - 1/2)$, 即 k_x 为非负的半整数.

此时, 一方面省略了预计算特征函数的步骤; 另一方面可以利用快速正弦变换, 快速余弦变换等技术减少复杂度; 并且, 此时不需要存储基函数, 只需要记录输入信号与各基函数的内积. 这一系列方法显著减少了算法的复杂度, 并减少了存储量.

Grid size	Basis dimension	Running Time					Total Memory Usage		
		on-the-fly basis (OTF)	cached basis	DCT (ours)	speedup vs. OTF	speedup vs. cached	cached basis	Ours	memory savings
128 ³	200	8.660 secs	1.65 secs	0.10 secs	87×	17×	10.2 GB	185 MB	55×
	1000	44.10 secs	9.54 secs	0.10 secs	440×	95×	50.5 GB	223 MB	226×
220 ³	200	45.56 secs	17.2 secs	0.78 secs	58×	22×	52.0 GB	938 MB	55×
	24000	6630 secs	-	0.78 secs	8499×	-	6.10 TB	26.0 GB	235×

图 2: QiaoDong Cui 等人得到的复杂度分析表 [7]

5 谱基函数稀疏化 (Sparsifying Spectral Bases)

全局特征函数的一个问题在于, 其分布非常广, 这导致最后得出的系统将是非常稠密的 (Dense), 一方面这会在进行线性代数计算降低计算效率并增加存储负担, 另一方面会使一个局部的输入对全局造成显著影响, 而这时是我们所不希望发生的. 因此, 有时我们需要对特征基函数进行稀疏化的操作.

Brandt 等人 [8] 在 2017 年利用 L1 范数进行了一个稀疏化的尝试, 他将特征向量的确定转化为了一系列优化问题, 这组优化问题的解在给定的范数下将是一个标准正交集, 并且使得给定的能量最小, 其中的能量函数与 L1 范数直接相关. 这一优化问题表述如下:

$$u_i := \begin{cases} \arg \min_{\mathbf{u}} \mathbf{u}^T H \mathbf{u} + \mu \|\mathbf{u}\|_1 \\ \text{subject to } \mathbf{u}^T M \mathbf{u} = 1 \text{ and } \forall j < i : \mathbf{u}^T M \mathbf{u}_j = 0 \end{cases}$$

, 其中 M 为质量矩阵 (Mass Matrix), 是一个对角阵; $\|\mathbf{u}\|_1 := \sum_{i=0}^N M_{ii} |u_i|$, 其中 M_{ii} 是 M 的第 i 个对角线元素, u_i 是 $\mathbf{u}$ 的第 i 个分量; H 是给定的能量函数 $E : \mathbb{R}^N$ 的 Hessian 矩阵, μ 是一个权重, 这一权重越大, 稀疏化的程度越高.

可以注意到, 当权重增大时, 决定 $\arg \min_{\mathbf{u}} \mathbf{u}^T H \mathbf{u} + \mu \|\mathbf{u}\|_1$ 时, $\|\mathbf{u}\|_1$ 的影响将变大, 这使得在此约束条件下的 $\mathbf{u}$ 将越来越难以变大, 从而将 $\mathbf{u}$ 稀疏化 (也就是将其中的一些项化为 0), 其代表的振动模态也将局域化.

当然, 这种稀疏化的操作也存在一些弊端, 比如会牺牲谱方法的收敛性, 从而需要更多的约束条件来达到所要求的精度; 并且稀疏化操作的计算也比较复杂, 需要更多的时间花费; 最后, 现在还没有技术可以对稀疏化的程度进行显性的控制, 也就是说, 没有办法控制某个参数来直接地控制稀疏化到什么程度, 这些都是稀疏化操作目前面临的问题.

另一种比较自然的稀疏化思路是, 进行域分解操作, 将整个区域划分为几块, 单独地求解各个区域上的特征向量. 但是这种操作也非常明显地会产生很多问题: 区域的划分必须细致的选择, 使特征函数在边界上能够满足一些给定的连续性条件, 否则最终的视觉效果将会非常糟糕; 但如果为了保证连续性, 选择的约束条件过多时, 有可能出现过度约束 (Overconstrain) 的情况, 使得整个系统陷入死锁, 无法运动.

杨垠等人 [9] 在 2013 年针对这个问题提出了一些新的方法, 也就是针对域分解后各分解区域内的基函数的改造.

他单独考虑了边界上的基函数, 将边界模式 (Boundary Mode) 分为两部分, 一部分是认为两区域交界面仅进行刚体运动的刚性边界模式 (Rigid Boundary Mode) Φ^R , 另一部分是通过合并两区域交界面非刚性位移引起的变形对刚性边界模式进行补充的软边界模式 (Soft Boundary Mode) Φ^S . 在求出这两部分从而确定边界基函数后, 再通过边界基函数驱动计算内部基函数.

他将边界模式定义为当从外部施加边界位移时, 在静态平衡状态下的整体的域位移. 这种平衡可以用 $K\mathbf{u} = \mathbf{f}$ 的形式表示, 其中 K 为域的刚度矩阵. 用 u 和 f 表示域的位移和力. 可以对 K 进行重排序与分块, 通过求解以下平衡来计算内部的未知响应变形 Φ_I :

$$\begin{bmatrix} K_{II} & K_{IB_1} & \cdots & K_{IB_k} \\ K_{B_1I} & K_{B_1B_1} & \cdots & K_{B_1B_k} \\ & & \ddots & \\ K_{B_kI} & K_{B_kB_1} & \cdots & K_{B_kB_k} \end{bmatrix} \begin{bmatrix} \Phi_I \\ \Phi_B \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{F}_B \end{bmatrix}$$

, 其中 I 指内部的点, 而 B_i 指第 i 个边界上的点, 而 Φ_B 将被划分为一个块对角阵

$$\Phi_B = \begin{bmatrix} \Phi_{B_1} & & & \\ & \Phi_{B_2} & & \\ & & \ddots & \\ & & & \Phi_{B_k} \end{bmatrix}$$

对于刚性边界模式 Φ^R , 依上述原则将被分解为 $\Phi^R = \begin{bmatrix} \Phi_I^R \\ \Phi_B^R \end{bmatrix}$, 其中 Φ_I^R 待求, 而 Φ_B^R 满足

$$\Phi_{p,p \in B}^R = [\mathbf{x}|\mathbf{y}|\mathbf{z}|(\mathbf{p} - \mathbf{c}_i) \times \mathbf{x}|(\mathbf{p} - \mathbf{c}_i) \times \mathbf{y}|(\mathbf{p} - \mathbf{c}_i) \times \mathbf{z}]$$

, 其中 $\mathbf{c}_i$ 为边界 B_i 的质心, 而 $\mathbf{x} = [1, 0, 0]^T$, $\mathbf{y} = [0, 1, 0]^T$, $\mathbf{z} = [0, 0, 1]^T$. 前三列 $\mathbf{x}, \mathbf{y}, \mathbf{z}$ 代表着三个线性独立的平移位移场, 是平移模式; 而后三列是旋转模式, 通过将沿三个线性独立旋转轴 (即 $\mathbf{x}$ 、 $\mathbf{y}$ 和 $\mathbf{z}$) 的旋转切线方向的位移分配给每个界面节点 (Interface Node) 来构建.

以此为基础, 可以计算出 Φ_I^R , 有

$$\Phi_I^R = -K_{II}^{-1} [K_{IB_1} \Phi_{B_1}^R | K_{IB_2} \Phi_{B_2}^R | \cdots | K_{IB_k} \Phi_{B_k}^R]$$

对于软边界模式 Φ^S , 同样地, 依上述原则将被分解为 $\Phi^S = \begin{bmatrix} \Phi_I^S \\ \Phi_B^S \end{bmatrix}$, 其中 Φ_I^S 待求, 而 Φ_B^S 满足

$$\Phi_B^S = H \otimes I$$

, 其中的 H 为 Valley 等人 [10] 在 2008 年提出的多谐基 (Manifold Harmonics Bases).

同样以此为基础, 可以计算出 Φ_I^S , 有

$$\Phi_I^S = -K_{II}^{-1} [K_{IB_1} \Phi_{B_1}^S | K_{IB_2} \Phi_{B_2}^S | \cdots | K_{IB_k} \Phi_{B_k}^S]$$

而对于内部基函数, 他们通过解以下关于内部自由度 (Internal DoFs) 的广义本征问题以通过边界基函数驱动计算内部基函数:

$$\begin{cases} (K_{II} - \Lambda M_{II}) \Phi_I^N = 0 \\ \Phi_B^N = 0 \end{cases}$$

这一方法显著提高了模拟的精度, 并且也保持了运算效率, 可以支持一些实时模拟.

6 向非线性问题拓展

实际上, 上述这些全局的基函数很难拓展到非线性问题上, 因为现有的线性模态基函数本质上是对原有的构形空间 (Configuration Space) 在原点处的线性展开, 因此这些基函数只能模拟一些小的形变, 在形变较大时, 由于不再能用线性的情况很好地模拟这些形变, 仅使用线性基函数就会出现一些问题, 比如会使物体的体积和内蕴几何发生改变.

Barbic 等人 [11] 在 2005 年的一些工作中, 考虑了这个问题. 他们考虑了一些线性基函数的导数 $\Phi^{ij} = \frac{\partial^2 u}{\partial p_i \partial p_j}$, 从而将更高阶的信息囊括入了降维操作的框架内. 其主要思想是, 在以通常方式计算出在各 p_i 上的方向导数 Ψ^i (这可以利用线性模态分析 (Linear Modal Analysis) 技术 [12] 完成). 在一般的做法中, 此时应当已有了

$$u(p) = \sum_{i=1}^k \Psi^i p_i + \mathcal{O}(p^2)$$

, 并以 $\{\Psi^i\}$ 作为分解结果. 但是 Barbic 等人提出, 可以认为

$$\Phi^{ij} = \frac{\partial}{\partial p_j} (\Psi^i (\Psi^l p_l))|_{p=0^k}$$

此时将有一个更好的插值模拟

$$u(p) = \sum_{i=1}^k \Psi^i p_i + \frac{1}{2} \sum_{i=1}^k \sum_{j=1}^k \Phi^{ij} p_i p_j + \mathcal{O}(p^3)$$

, 从而提高对各种非线性问题的适应性.

在实际操作中, 往往不直接使用上述公式计算 Φ^{ij} , 而是解线性方程组

$$K \Phi^{ij} = -(H : \Psi^i) \Psi^j$$

, 其中 H 为 Hessian 刚度张量, 表示为

$$H_{ijl} = \frac{\partial}{\partial u_l} (K_{ij}(u))|_{u=0^{3n}}, i, j, l = 1, 2, \dots, 3n$$

, $(H : a)$ (其中 a 为一向量 $a^l e_l$) 为一个矩阵, 其 (i, j) 位置的元素为 $H_{ijl} a^l$, 其中 $i, j = 1, 2, \dots, 3n$.

当然, 非常显然地, 这些附加的操作提高了算法的复杂度, 在同等条件下显著增加了算法的运行时间, 可以看到, 随着 k 的增加, 相比于线性模式的

运算时间 (左起第三列), 计算二次项的时间 (左起第四, 第五列) 是其数倍甚至数十倍.

Model	k	Compute linear modes	Build right-hand sides of Eq. 14	Solve Eq. 14
spoon	6	24 sec	6.5 sec	33 sec
tower	20	65 sec	226 sec	26 min
heart	20	111 sec	291 sec	28 min

图 3: Barbic 等人得到的复杂度分析表 [11]

黄劲等人 [13] 在 2011 年也借助 $SO(3)$ 旋转群和其上的李代数理论做了一些尝试, 从而一定程度上解决了旋转应变中的体积膨胀问题.

这实际上是对线性模拟弹性形变的结果 (也就是未做改进时的结果) 的一个改进, 从而消除其变形 (Deformation) 后的体积膨胀.

认为模型的静止状态位置为 r , 变形后的位置为 x , 则形变为 $u = x - r$, u 可以用一般的计算方法对

$$Ku + D\dot{u} + M\ddot{u} = 0$$

分析获得. 在得到初步结果 u 之后, 算法分为以下几步来改进结果:

首先对结果 u 的梯度 ∇u 做分解

$$\nabla u = \frac{\nabla u + \nabla u^T}{2} + \frac{\nabla u - \nabla u^T}{2}$$

该式中的第一部分 $\frac{\nabla u + \nabla u^T}{2}$ 是对称的, 代表着原点处的小形变 $\epsilon(u)$, 第二部分 $\frac{\nabla u - \nabla u^T}{2}$ 是反对称的, 代表着原点处的小旋转 $\omega(u)$.

然后解系统的广义特征值问题

$$Ky = \lambda^2 My$$

, 利用特征频率 λ_i 构造模态频率矩阵 (Modal Frequency Matrix) $\Lambda = \text{diag}(\lambda_1^2, \lambda_2^2, \dots, \lambda_n^2)$, 再以此求解模态位移矩阵 (Modal Displacement Matrix) $W = (W_1 W_2 \dots W_n)$. 求解这一矩阵只需要解两个方程

$$W^T K W = \Lambda, W^T M W = I$$

接下来用一个指数映射分别对每个模态位移维度 W_i 上的梯度进行修正, 得到新的梯度 $(\nabla u')_i$. 具体而言, 就是

$$(\nabla u')_i = R_i S_i - I = \exp(\omega(W_i))(I + \epsilon(W_i)) - I$$

, 这个结果就是我们认为各个维度上”最正确的”变形的梯度, 此时 $\nabla u'$ 仅为一个符号, 不代指某一具体的 u' 的梯度.

最后借助各维度修正后的梯度重建整体的变形, 这只需要解决一个 Poisson 问题

$$\begin{aligned} \arg \min_X \sum_i \|(\nabla X)_i - \exp(\omega(W_i))(\epsilon(W_i) + I)\|^2 \\ \text{subject to } \frac{1}{n} \sum_{i=1}^n X_i = c \end{aligned}$$

. 其中 $\|\cdot\|$ 取矩阵的 Frobenius 范数, c 的取值并不是一定的, 一般可以取两个边界形状的线性插值, X 即是我们得到的最后结果.

这个结果可以有效解决变形后的体积膨胀问题, 但是也存在着一些局限性. 首先, 这一算法是从纯几何的角度进行的后处理, 因此完全没有考虑到材质信息, 也没有考虑到物理上的真实性, 这主要是因为旋转和形变并不是完全独立的, 而是需要满足一些可积性条件, 否则会使得解 Poisson 问题的过程中出现一些误差, 导致物体的视觉效果比较”软”或者旋转的频率上不合理.

7 前沿问题展望

目前大多数的降维方法都没有考虑到时间轴的影响因素. 降维方法本质上是一种近似, 随着时间轴向前运行, 时间靠后的模拟需要依赖时间靠前的模拟结果, 而时间靠前的模拟结果本就存在误差, 这样就会使误差不断累积, 形成一个随时间增长的误差, 这就需要我们进行时空联合降维 (Space-Time Reduction). 在这种降维思路中, 我们首先需要考虑, 降维时应当尽量保持哪些物理量, 或者说以什么作为能量. 保持动能或者势能 (这是一般情况下的做法) 在这一新降维思路下也许并不适用, 目前的一个想法是利用系统的 Lagrange 量作为能量, 或以 Lagrange 量为基础构造其他形式的能量.

另一个重要的领域是非线性问题, 虽然上文中提到了一些解决方案, 但它们都有复杂度过高, 或者模拟效果不佳的问题. 一些研究者提出了另一种完全不同的思路, 试图通过机器学习的方法令计算机自主寻找合适的基函数. 比如 Owhadi 等人 [14] 在 2017 年就利用博弈论的方法提出一种新的解决思路, 算法使两个 AI 参与者提出两组不同的基函数, PlayerI 提出 u^I , PlayerII 提出 u^{II} , 以 $\frac{\|u^I - u^{II}(U^I)\|}{\|u^I\|}$ 为标准, PlayerI 使该函数取值最大化, PlayerII 使该函数取值最小化, 如此循环, 最后得到一组比较好的基函数.

利用机器学习的方式寻找基函数是一种新的思路, 而近年来生成对抗网络 (GAN) 的发展使得机器学习寻找基函数有了更加有力的工具.

最后, 还有一个重要的领域, 就是对数值解法的定量研究. 目前的大多数论文都不可避免地需要在模拟精度与运行时间中做取舍, 这种经验式的研究无法给出一个统一的标准去说明哪种方法是更优的, 这在很大程度上影响了计算方法的发展. 因此, 虽然看上去并不是一个亟待解决的问题, 但寻找一个衡量标准或者说寻找一个关于基函数性能的不等式 (类似于不确定性原理) 是一个非常重要的问题.

8 结论与总结

我们简要介绍了谱方法降维的发展过程以及优化方向. 可以看到, 谱方法具有很多方便处理的优点, 也有不少限制其发展的缺点.

谱方法具有良好的谱收敛性, 这意味着仅仅数十个或者数百个基函数就可以有良好的模拟效果; 同时谱方法的运行时间复杂度只需要考虑基的数量, 而不需要考虑网格大小, 对较大的网格也不易出现运行时间的爆炸; 最后, 由于其理论的普适性, 谱方法对系统的适应性很强, 因此应用范围非常广. 这些都是谱方法的优点.

同样在以上的简述中也可以看到谱方法的缺点. 在谱方法的计算中, 并没有空间自由度的概念, 这使得我们的运算结果并不与空间中的位置相依附; 其次, 谱方法需要较长的预计算时间, 预计算的时间复杂度甚至往往不是线性的, 而是以多项式级别增长; 并且谱方法的全局适应性也需要更大的存储空间以及更复杂的运算; 最后, 谱方法对非线性系统的适应性是有限的, 非线性系统中的谱方法应用直到现在也并没有完全解决, 这些都是至今为止的研究重点.

最后, 我们讨论了近年来一些前沿问题的发展方向, 在对精度的要求越来越高的情况下, 时间误差不得不纳入考虑, 时空联合降维的思路应运而生; 非线性问题仍然没有得到完全解决, 新兴的机器学习, 生成对抗网络等工具提供了新的解决思路; 除这些之外, 对数值解法进行定量研究的重要性也在不断凸显. 领域前沿的研究仍然充满着诸多难题.

参考文献

- [1] A. Pentland and J. Williams, “Good vibrations: model dynamics for graphics and animation,” in *Proceedings of the 16th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1989*, 1989.
- [2] C. Schulz, C. V. Tycowicz, H. P. Seidel, and K. Hildebrandt, “Animating deformable objects using sparse spacetime constraints,” *Acm Transactions on Graphics*, vol. 33, no. 4, p. 109, 2014.
- [3] S. Li, J. Huang, F. D. Goes, X. Jin, H. Bao, and M. Desbrun, “Space-time editing of elastic motion through material optimization and reduction,” *Acm Transactions on Graphics*, vol. 33, no. 4, pp. 1–10, 2014.
- [4] D. James, J. Barbic, and D. Pai, “Precomputed acoustic transfer: Output-sensitive, accurate sound generation for geometrically complex vibration sources,” *Acm Transactions on Graphics*, vol. 25, no. 3, pp. 987–995, 2006.
- [5] X. U. Hongyi, L. I. Yijing, Y. Chen, and J. Barbic, “Interactive material design using model reduction,” *Acm Transactions on Graphics*, vol. 34, no. 2, pp. 1–14, 2015.
- [6] T. D. Witt, C. Lessig, and E. Fiume, “Fluid simulation using laplacian eigenfunctions,” *ACM Transactions on Graphics (TOG)*, vol. 31, no. 4CD, pp. 10.1–10.11, 2012.
- [7] Q. Cui, P. Sen, and T. Kim, “Scalable laplacian eigenfluids,” *ACM Transactions on Graphics*, vol. 37, no. 4CD, pp. 1–12, 2018.
- [8] C. Brandt and K. Hildebrandt, “Compressed vibration modes of elastic bodies,” *Computer Aided Geometric Design*, vol. 52-53, no. MAR.-APR., pp. 297–312, 2017.
- [9] Y. Yang, W. Xu, X. Guo, K. Zhou, and B. Guo, “Boundary-aware multidomain subspace deformation,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 10, pp. 1633–1645, 2013.

- [10] B. Vallet and B. Levy, “Spectral geometry processing with manifold harmonics,” *Computer Graphics Forum (Proceedings Eurographics)*, vol. 27, no. 2, pp. 251–260, 2008.
- [11] J. Barbic and D. James, “Real-time subspace integration for st. venant-kirchhoff deformable models,” *ACM Transactions on Graphics (TOG)*, vol. 24, no. 3, pp. 982–990, 2005.
- [12] Shabana and A. Ahmed, *Theory of Vibration Volume II: Discrete and Continuous Systems* /. Discrete and continuous systems, 1990.
- [13] H. Jin, Y. Tong, K. Zhou, H. Bao, and M. Desbrun, “Interactive shape interpolation through controllable dynamic deformation,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 7, pp. 983–992, 2011.
- [14] H. Owhadi and C. Scovel, “Universal scalable robust solvers from computational information games and fast eigenspace adapted multiresolution analysis,” *arXiv e-prints*, 2017.